

Python Assignment 4

Question 1 :

What is the difference between List and Tuple in Python?

Explain in terms of:

- Mutability
- Memory
- Performance
- Use cases

Answer :

List is a mutable, memory-consuming, and flexible collection used for dynamic data, whereas Tuple is an immutable, memory-efficient, and faster collection used for fixed data.

Feature	List	Tuple
Syntax	[]	()
Mutability	Mutable (can be changed)	Immutable (cannot be changed)
Memory Usage	Uses more memory	Uses less memory
Performance	Slightly slower	Slightly faster
Use Case	Dynamic data that may change	Fixed data that should not change

```
"""What is the difference between List and Tuple in Python?
Explain in terms of:
• Mutability
• Memory
• Performance
• Use cases
"""

import sys, timeit
print("-----List-----")
print('''A list is an ordered collection used to store multiple values in
a single variable, and it allows modification after creation.\n''')

lst = [10, 20.50, True, "Amar"]          #mixed data type allowed
print("Our List :", lst)                  #Ordered collection
print("Data type of list :", type(lst))   #type is list
print("Memory size of list :", sys.getsizeof(lst)) #Use more memory
lst[0] = 100                             #Mutable(can Modified)
lst.append(40)                            #Use case - modification is allowed
lst.remove("Amar")                        #Deletion is Allowed
lst_time=timeit.timeit("[10,20.50]",number=1000000)#Measure execution time
print("Execution time is :", lst_time)    #Slower
```

```

print("\n-----Tuple-----")
print("A tuple is similar to a list but is used when the data should not
be changed after creation.\n")
tpl = (10,20.50,True,"Amar")
print("Our Tuple : ", tpl)
print("Data Type of tuple : ", type(tpl))
print("Memory Size of tuple : ", sys.getsizeof(tpl))
#tpl[0] = 100 #Unmutable (cannot Modified)
#tpl.append(40) #Use case - modification is not allowed
#tpl.remove = ("Amar") #Deletion also not allowed
tpl_time = timeit.timeit("[10,20.50]", number = 1000000)
#Measure Execution Time
print("Execution Time is : ", tpl_time) #Faster

"""Note : timeit() is used to measure the execution time of a small piece
of Python code.
It helps compare the performance of different code snippets."""

```

Question2 :

Why are tuples faster than lists?

In which real-world scenario would you prefer a tuple over a list?

Answer :

```

"""Why are tuples faster than lists?
In which real-world scenario would you prefer a tuple over a list?
"""

print("\n Both tuple and list store multiple values, but tuples are" \
"generally faster because tuples are immutable, fixed size compare to list
has"
"Append, update, insert, delete operation and that operation dynamically
increased memory.\n")

print("Real life scenario use in below case")
location = (18.5204, 73.8567) #logitude and latitude are fixed value
dob = (15, 8, 1995) #Date of Birth is fixed not changed dynamically
employee = (101, "Amar", "Pune")
#employee name and id is fixed not change dynamically

print("Location : ",location)
print("DOB : ", dob)
print("Employee Primary Details : ", employee)

```

Question 3:

Predict the output:

lst = [10, 20, 30]

tpl = (10, 20, 30)

lst[0] = 100

tpl[0] = 100

Which line will raise an error and why?

Answer :

```
lst = [10, 20, 30]
tpl = (10, 20, 30)
lst[0] = 100
tpl[0] = 100      #This line generates error because tuple is non mutable
```

Question 4:

Explain why strings are immutable in Python.

What happens internally when you modify a string variable?

Answer :

```
s = "hello"
print(id(s))          # 2277229153376

s = s + " world"
print(id(s))          # 2277229168368

#DIFFERENT id mean allocate new object entirely, hence prove string is
immutable

print("-----internal operations-----")
"""Step 1:  s = "hello"
    Memory: [addr_1] → "hello"
    s points to addr_1

Step 2:  s = s + " world"
    Python reads "hello" from addr_1
    Allocates NEW memory [addr_2] → "hello world"
    s now points to addr_2

    [addr_1] "hello" → still exists,
    no longer referenced garbage collector will clean it up
"""
```

Question 5:

Predict the output:

```
s = "Python"
```

```
print(id(s))
```

```
s = s + "3"
```

```
print(id(s))
```

Explain the reason for change / no change in id().

Answer :

```
s = "Python"
```

```
print(id(s))
```

```
s = s + "3"
```

```
print(id(s))
```

#in python if variables value update then python PVM allocate new memory space while

#address changed old address memory get pointless and garbage collector release memory space

Questions 6:

What is a dictionary in Python?

Explain:

- Key–value pair
- Why keys must be immutable
- Why duplicate keys are not allowed

Answer :

```
student = {"name" : "Amar Bhandare",      #str -> str
           "age" : 29,                    #str -> int
           "birth_date" : "10-02-1996",    #str -> str
           "marks" : [80,70,40,20]}
```

```
print(student)    # {'name': 'Amar Bhandare', 'age': 29}
```

```
print(type(student))    # <class 'dict'>
```

```
print(type(student.values()))    # <class 'dict_values'>
```

Property	Rule	Reason
Keys	Must be immutable	Hashing requires a stable, unchanging value
Keys	Must be unique	Hash table maps one key → one memory slot
Values	Can be any type	No hashing needed; just stored data
Values	Can be duplicated	Two keys can point to the same value — no conflict

Question 7:

Predict the output:

```
d = {1: "One", 1: "ONE", 2: "Two"}
```

```
print(d)
```

Why does this happen?

Answer :

```
d = {1: "One", 1: "ONE", 2: "Two"}
print(d)
```

OUTPUT :

```
{1: 'ONE', 2: 'Two'}
```

Here 3 key value pair in dictionary, 1st two key is same but values are different and 3rd key value is unique, according to dict rules key are unique and values allowed to duplicates

Question 8:

What is the range data type in Python?

How is it different from a list of numbers?

Answer :

range is a built-in immutable sequence type that represents a sequence of numbers. It doesn't store all the numbers in memory it generates them on demand. Commonly use in loops in iteration.

Feature	range	list
Memory	Fixed (stores only start, stop, step)	Grows with size
Speed to create	Instant	Slower for large sizes
Mutability	Immutable	Mutable
Element access	Supports indexing & slicing	Supports indexing & slicing
Iteration	Yes	Yes
Membership test (in)	O(1) — instant	O(n) — scans the list
Can store non-integers	No — integers only	Yes — any type

When to use which?

Use **range** when you just need to iterate or count — for i in range(10):

Use **list** when you need to store, modify, or mix different values

In short, range is a **lazy**, memory-efficient sequence — it *describes* numbers without *storing* them.

```
import sys
```

```
a = range(5)
```

```
b = range(2, 8)
```

```
c = range(0, 10, 2)
```

```
d = range(10, 0, -1)
```

```

print("\nlist of A :",list(a))    # 0, 1, 2, 3, 4
print("list of B :",list(b))     # 2, 3, 4, 5, 6, 7
print("list of C :",list(c))     # 0, 2, 4, 6, 8
print("list of D :",list(d))     # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

print("\n",type(a))  #<class 'range'>
print(type(b))      #<class 'range'>
print(type(c))      #<class 'range'>
print(type(d))      #<class 'range'>

r = range(1_000_000)
l = list(range(1_000_000))

print("\nmemory size of r : ",sys.getsizeof(r),"Bytes")
l_size = print("memory size of l : ",sys.getsizeof(l),"Bytes")
in_MB = sys.getsizeof(l)
print(in_MB/(1024*1024),"MB")

```

Question 9:

9. Predict the output:

```

r = range(5)
print(r)
print(list(r))

```

Answer :

```

r = range(5)
print(r)          #range(0, 5)
print(list(r))    #[0, 1, 2, 3, 4]

```

Question 10:

What is the difference between:

range(1, 10)

range(1, 10, 2)

Explain the parameters.

Answer :

```

a = range(1, 10)      #(Start, Stop, Default 1)
b = range(1, 10, 2)   #(Start, Stop, Steps 2)

print(list(a))        #[1, 2, 3, 4, 5, 6, 7, 8, 9]
print(list(b))        #[1, 3, 5, 7, 9]

```